```
# URA302 Minor Projects
# Dr. Rohit Kumar Singla
# Thapar Institute of Engineering and Technology
# Department of Mechanical Engineering

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from queue import PriorityQueue

# ============================================================
# PROJECT 1: TIC-TAC-TOE GAME
# ============================================================

def print_board(board):
    print("\n")
    for i in range(0, 9, 3):
        print(" ", board[i], "|", board[i+1], "|", board[i+2])
        if i < 6:
            print(" ---+---+---")
    print("\n")



def check_winner(board, player):
    win_positions = [
        [0,1,2], [3,4,5], [6,7,8],
        [0,3,6], [1,4,7], [2,5,8],
        [0,4,8], [2,4,6]
    ]
    for pos in win_positions:
        if board[pos[0]] == board[pos[1]] == board[pos[2]] == player:
```

```
            return True
    return False



def check_tie(board):
    return " " not in board



def get_player_input(player, board):
    while True:
        try:
            pos = int(input(f"Player {player}, enter position (1-9): ")) - 1
            if pos < 0 or pos > 8:
                print("Invalid position! Choose 1-9.")
            elif board[pos] != " ":
                print("Position already taken!")
            else:
                return pos
        except ValueError:
            print("Please enter a valid number.")



def play_tic_tac_toe():
    board = [" "] * 9
    current_player = "X"

    while True:
        print_board(board)
        move = get_player_input(current_player, board)
        board[move] = current_player
```

```
        if check_winner(board, current_player):
            print_board(board)
            print(f"🎉 Player {current_player} wins!")
            break

        if check_tie(board):
            print_board(board)
            print("🤝 It's a tie!")
            break

        current_player = "O" if current_player == "X" else "X"

# ============================================================
# PROJECT 2: TO-DO LIST APPLICATION
# ============================================================

def add_task(tasks):
    task = input("Enter new task: ")
    tasks.append(task)
    print("Task added successfully!")


def view_tasks(tasks):
    if not tasks:
        print("No tasks available.")
    else:
        print("\nYour Tasks:")
        for i, task in enumerate(tasks):
            print(f"{i}. {task}")
```

```
def delete_task(tasks):
    try:
        index = int(input("Enter task index to delete: "))
        if 0 <= index < len(tasks):
            removed = tasks.pop(index)
            print(f"Task '{removed}' deleted.")
        else:
            print("Invalid index.")
    except ValueError:
        print("Please enter a valid number.")



def to_do_list_app():
    tasks = []
    while True:
        print("\n--- TO-DO LIST MENU ---")
        print("1. Add Task")
        print("2. View Tasks")
        print("3. Delete Task")
        print("4. Exit")

        choice = input("Enter choice: ")

        if choice == "1":
            add_task(tasks)
        elif choice == "2":
            view_tasks(tasks)
        elif choice == "3":
            delete_task(tasks)
        elif choice == "4":
            break
```

```
        else:
            print("Invalid choice.")

# ============================================================
# PROJECT 3: ROBOT PATH PLANNING (A* ALGORITHM)
# ============================================================

def heuristic(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])



def a_star(grid, start, goal):
    pq = PriorityQueue()
    pq.put((0, start))
    came_from = {}
    g_cost = {start: 0}

    while not pq.empty():
        _, current = pq.get()

        if current == goal:
            path = []
            while current in came_from:
                path.append(current)
                current = came_from[current]
            path.append(start)
            return path[::-1]

        for dx, dy in [(-1,0),(1,0),(0,-1),(0,1)]:
            neighbor = (current[0] + dx, current[1] + dy)
```

```
        if (0 <= neighbor[0] < grid.shape[0] and
            0 <= neighbor[1] < grid.shape[1] and
            grid[neighbor] == 0):

            new_cost = g_cost[current] + 1
            if neighbor not in g_cost or new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                f_cost = new_cost + heuristic(neighbor, goal)
                pq.put((f_cost, neighbor))
                came_from[neighbor] = current
    return None


def robot_path_planning():
    rows = int(input("Enter number of rows: "))
    cols = int(input("Enter number of columns: "))
    grid = np.zeros((rows, cols), dtype=int)

    obs = int(input("Enter number of obstacles: "))
    for _ in range(obs):
        r, c = map(int, input("Enter obstacle position (row col): ").split())
        grid[r][c] = 1

    start = tuple(map(int, input("Enter start position (row col): ").split()))
    goal = tuple(map(int, input("Enter destination (row col): ").split()))

    print("\nGrid (Before Pathfinding):")
    print(pd.DataFrame(grid))

    path = a_star(grid, start, goal)
```

```
    if path is None:
        print("No valid path found.")
        return

    for r, c in path:
        grid[r][c] = 2

    print("\nGrid (After Pathfinding):")
    print(pd.DataFrame(grid))

    plt.imshow(grid, cmap="gray")
    plt.scatter(start[1], start[0], c="green", label="Start")
    plt.scatter(goal[1], goal[0], c="blue", label="Destination")

    path_y = [p[0] for p in path]
    path_x = [p[1] for p in path]
    plt.plot(path_x, path_y, 'r.', label="Path")

    plt.legend()
    plt.title("Robot Path Planning using A*")
    plt.show()

# ============================================================
# MAIN MENU (SINGLE TERMINAL)
# ============================================================

def main():
    while True:
        print("\n========== URA302 MINOR PROJECTS ==========")
        print("1. Tic-Tac-Toe Game")
        print("2. To-Do List Application")
```

```
        print("3. Robot Path Planning (A*)")
        print("4. Exit")

        choice = input("Select Project: ")

        if choice == "1":
            play_tic_tac_toe()
        elif choice == "2":
            to_do_list_app()
        elif choice == "3":
            robot_path_planning()
        elif choice == "4":
            print("Exiting program.")
            break
        else:
            print("Invalid choice.")


if __name__ == "__main__":
    main()
```